

Mail-Server-Factory

Run your mail server like the boss — describe it in JSON, deploy it anywhere.

Summary

Mail Server Factory is a production-ready, automated mail-server provisioning tool. The user writes a simple JSON configuration; the Factory interprets it and performs all installations and initializations on the target OS, deploying a Docker-based, loosely-coupled mail stack across 12 connection types.

Short description

A Kotlin/Shell tool that turns a JSON description into a fully installed, Dockerized mail server. It supports 12 connection types (SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt, and more), a full security framework, 25 Linux distributions, and ships with 439 passing tests.

Long description

Standing up a real, secure mail server is one of the classic rites of passage in systems administration — and one of the most reliably miserable. Postfix, Dovecot, TLS certificates, DNS records, firewall rules, and per-distribution quirks all have to line up perfectly, and a single wrong directive means silently bounced mail or an open relay. Mail Server Factory takes that entire body of hard-won, error-prone expertise and captures it in software. Instead of hand-configuring each piece across an unfamiliar OS, the end-user writes the desired outcome as a simple JSON document; the Factory reads that JSON and executes the exact installation and initialization steps required on the target operating system, standing up a mail stack that runs on Docker with every component loosely coupled — a design choice that keeps the stack horizontally scalable and lets any single component be upgraded or replaced in isolation. And it is deliberately reach-agnostic: 12 connection types let the identical

tool and identical JSON target a local machine, a remote host over SSH, a Docker or Kubernetes runtime, cloud instances via AWS SSM / Azure Serial Console / GCP OS Login, or VMs via Libvirt — the same declarative description, provisioned wherever you point it. It supports 25 Linux distributions across Western (Ubuntu, Debian, CentOS, Fedora, AlmaLinux, Rocky, openSUSE), Russian (ALT, Astra, ROSA), and Chinese (openEuler, openKylin, Deepin) families, with unattended installation via preseed/kickstart/cloud-init/autoyast and QEMU-based VM automation for testing. Enterprise features are extensive: AES-256-GCM encryption, enforced password and SSH-key policies, automated firewall configuration for mail ports (25/587/465/993/995), TLS/SSL with certificate validation and HSTS, audit logging, and RBAC. Operational features include JVM tuning (G1GC), Caffeine caching, connection pooling, Prometheus-compatible metrics, structured logging, hot-reloading configuration, and secrets management. The project reports 439 tests at 100% pass and a clean SonarQube quality gate. It is the flagship of the Server-Factory org.

Why we built it

Setting up a secure, production mail server is notoriously error-prone and OS-specific. Mail Server Factory captures that expertise in a declarative JSON model plus an execution engine, so a correct, secured, Dockerized mail stack can be reproduced on any supported target without manual step-by-step work.

Why it's a game-changer

It demotes mail-server provisioning from a specialist, days-long, get-it-exactly-right ordeal to an act of writing configuration — then makes that act portable across 12 connection types and 25 Linux distributions and hardened by enterprise security defaults out of the box. The result is reproducible and *verifiable*: the same JSON yields the same secured stack every time, and the project's reported 439 passing tests and clean SonarQube gate mean the engine doing the work is itself held to account rather than trusted on reputation.

What's innovative

- Declarative JSON → interpreted installation/initialization on the target OS.
- 12 connection types (local, SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt, and more) behind one tool.
- 25-distribution support with unattended install (preseed/kickstart/cloud-init/autoyast) and QEMU automation.

- Loosely-coupled Dockerized stack for independent scaling/upgrades.

Challenges & solutions

- **OS/distribution heterogeneity:** solved with per-distribution recipes, unattended-install configs, and QEMU-based cross-distro testing.
- **Reaching many deploy targets:** solved with 12 pluggable connection types under a common installation engine.
- **Security by default:** solved with AES-256-GCM, enforced key/password policies, automated firewall rules, and TLS/HSTS.
- **Trust in correctness:** solved with a 439-test suite (100% passing) and a clean SonarQube gate.

Tech stack (why + how)

- **Kotlin** — the Factory engine and installation-step logic (179K bytes; Kotlin 2.0.21).
- **Shell** — provisioning scripts, ISO/QEMU managers, and OS automation (dominant by bytes).
- **Docker** — the runtime for the deployed, loosely-coupled mail stack.
- **QEMU** — VM automation for cross-distribution installation and testing.
- **JSON** — the user-facing declarative configuration format.
- **Gradle 8.14.3 / Java 17** — build toolchain.
- **Caffeine** — multi-region caching; **G1GC-tuned JVM** for performance.
- **Prometheus-compatible metrics** — monitoring; Grafana/ELK-ready.
- **Sieve** — mail filtering rules (small footprint in language stats).

Note: GitHub marks the repository as a fork within the Server-Factory org. Predates the AI product line; presented as a mature DevOps/provisioning flagship, not an AI utility.